

XII DAYS

Documentation technique exhaustive

Version mod : 1.5.42

Minecraft 1.21.1 - NeoForge 21.1.219

Package de base : com.mceteams.xiiday

Document genere le 09/05/2026

Ce document recense l'integralite des modifications, reorganisations, creations et decisions d'architecture prises au cours du developpement du mod XII Days.

Table des matieres

- 1 Reorganisation des paquetages
- 2 Architecture reseau
- 3 Systeme de fin de partie
- 4 Systeme de spectateur
- 5 Popups de points
- 6 Totem de Revivality
- 7 Ecrans et interface
- 8 Commandes
- 9 Gestion des donnees
- 10 Corrections de bugs
- 11 Decisions d'architecture
- 12 Liste exhaustive des fichiers

1. Reorganisation des paquetages

La structure initiale etait organisee par type de fichier (blocks/, utils/, menus/) sans lien avec le domaine fonctionnel. La refonte a consiste a regrouper les classes par domaine metier.

1.1 Structure avant

```
src/main/java/com/mceteams/xiideays/
  blocks/      (TeamCoreBlock, SpawnerBlock, CoreBlockEntity...)
  utils/       (DataManager.java)
  menus/       (ScoreboardScreen, EndScoreboardScreen...)
  commands/    (XiideaysCommand.java)
  items/       (ItemRegistry.java)
  network/     (OpenScoreboardPacket.java...)
  Xiideays.java
```

1.2 Structure apres

```
src/main/java/com/mceteams/xiideays/
  world/       (BlockRegistry, BlockEntityRegistry, CoreBlockEntity,
  |            SpawnerBlockEntity, teamCore, teamSpawner -> renommes)
  game/        (TeamManager, DaysManager, PointsManager, ScoreboardManager,
  |            PointType, TaskScheduler)
  spectator/    (SpectateManager, SpectatePackets)
  screen/      (ScoreboardScreen, EndScoreboardScreen, TeamStatsScreen,
  |            MenuRegistry)
  network/     (OpenScoreboardPacket, OpenEndScoreboardPacket,
  |            PointsPopupPayload, PacketHandler, ClientScoreboardHandler,
  |            ClientEndScoreboardHandler, PointsPopupClientHandler,
  |            CoreMaze*, RequestTeamStatsPacket, OpenTeamStatsPacket...)
  data/        (DataManager, TeamData, TeamStatsData, PlayerStatsData,
  |            DayCycleData)
  item/        (ItemRegistry, TotemRevivalityItem, CoreDestroyerItem...)
  command/     (CommandRegistry, XiideaysCommand)
  player/      (PlayersHandler, ClientRankTracker)
  config/      (Config)
  restriction/ (RestrictionsManager)
  visual/      (ZoneVisualizer)
  client/      (ClientModEvents - NOUVEAU)
```

1.3 Renommages

Les classes en minuscules ont ete renommees en PascalCase pour respecter les conventions Java :

- teamCore -> CoreBlockEntity
- teamSpawner -> SpawnerBlockEntity
- teamCoreBlock -> CoreBlock
- teamSpawnerBlock -> SpawnerBlock

1.4 Fichiers supprimes

- SpectatorZoneCommand.java (remplace par /xiideays zone)
- DataManager.java duplique dans utils/ (un seul dans data/)
- Anciens repertoires vides (blocks/, utils/, menus/, items/ -> devenus world/, data/, screen/, item/)

Note : Cette reorganisation etait purement structurelle : aucune interface publique ni signature de methode n'a ete modifiee.

2. Architecture reseau

2.1 Principe general

Le reseau suit le systeme de CustomPacketPayload de NeoForge 1.21. Chaque paquet est un record immutable implementant CustomPacketPayload, avec un StreamCodec<FriendlyByteBuf> pour la serialisation manuelle. Les paquets sont enregistres dans PacketHandler via registerPayloadHandlersEvent.

2.2 Paquets S2C (Serveur -> Client)

OpenScoreboardPacket	-> ouvre ScoreboardScreen
OpenEndScoreboardPacket	-> ouvre EndScoreboardScreen (NOUVEAU)
PointsPopupPayload	-> affiche popup de points (NOUVEAU)
OpenTeamStatsPacket	-> ouvre TeamStatsScreen
CoreMazeOpenPacket	-> ouvre le maze

2.3 Paquets C2S (Client -> Serveur)

RequestScoreboardPacket	-> demande les donnees du scoreboard
RequestTeamStatsPacket	-> demande les stats d'equipe
CoreMazeAnswerPacket	-> reponse du maze
SpectateSwitchPayload	-> changement de cible spectateur

2.4 OpenEndScoreboardPacket (NOUVEAU)

Payload de fin de partie contenant toutes les donnees necessaires a l'affichage du classement final. Le serveur pre-calcule le MVP et les statistiques de toutes les equipes avant d'envoyer le paquet.

```
public record OpenEndScoreboardPacket(
    String winningTeamName,
    MvpData mvp,                // playerName, playerUUID,
    List<TeamEntry> teamStats    // teamName, points, kills, deaths,
)                               // blocksMined, damageDealt, finalPosition
```

StreamCodec manuel : encode/decode champ par champ. Le handler cote client (ClientEndScoreboardHandler) ouvre EndScoreboardScreen avec les donnees.

2.5 PointsPopupPayload (NOUVEAU)

Petit paquet leger pour notifier le joueur d'un gain/perte de points. Le handler cote client alimente la file de popups animees.

```
public record PointsPopupPayload(int points, String typeName)
```

2.6 ClientModEvents (NOUVEAU)

Classe annotee @EventBusSubscriber(value = Dist.CLIENT) qui enregistre un calque GUI via RegisterGuiLayersEvent pour afficher les popups de points en surimpression du jeu.

```
event.registerAboveAll(
    ResourceLocation.fromNamespaceAndPath(MODID, "points_popup"),
    PointsPopupClientHandler::render
);
```

Note : Le LayeredDraw.Layer en 1.21.1 a pour signature (GuiGraphics, DeltaTracker) et non (GuiGraphics, float).

3. Systeme de fin de partie

Le systeme de fin de partie corrige 4 bugs relies : (1) ecran de fin non affiche, (2) jour non arrete, (3) jour relancable apres fin, (4) totem sans attribution de points.

3.1 Detection de la fin

Dans `TeamManager.eliminateTeam()`, apres elimination, on compte les equipes restantes. Quand il n'en reste qu'une :

```
if (remainingTeams == 1) {
    setWinningTeam(winnerTeam);    // finalPosition = 1
    DaysManager.stop();            // arret du jour
    PacketDistributor.sendToAllPlayers(
        buildEndPacket(winnerTeam)); // envoi ecran de fin
}
```

3.2 Calcul des donnees de fin (buildEndPacket)

Le serveur calcule toutes les donnees cote serveur (ou `DataManager` est accessible) :

- MVP : joueur avec le plus grand `team_points` dans `PlayerStatsData`, parcourt toutes les equipes et tous les membres
- Stats equipe : points, kills, blocs mines, degats infliges lus depuis `TeamStatsData`
- Morts equipe : somme des `PlayerStatsData.getDeaths()` de chaque membre (`TeamStatsData` ne stocke pas les morts)
- Classement trie par `finalPosition`

3.3 Blocage du redemarrage

Dans `DaysManager.start()`, verification en tete de methode :

```
if (TeamManager.isGameOver()) {
    // Message d'erreur et return false
}
```

`isGameOver()` retourne vrai si `countRemainingTeams() <= 1` ou si `getWinningTeam() != null` (une equipe a `finalPosition == 1`).

3.4 Reanimation d'equipe (reviveTeam)

Nouvelle methode dans `TeamManager` pour permettre a un admin de rehabiliter une equipe eliminee :

- Remet `eliminated = false`
- Reset `finalPosition = 0` pour l'equipe reanimée
- Reset `finalPosition = 0` pour toutes les equipes marquees gagnantes (`finalPosition == 1`)
- Le jeu peut reprendre normalement (jours relancables)

3.5 DaysManager.stop() - Refactoring

La methode `stop()` existante prenait un `CommandContext` en parametre. Elle a ete refactorée en deux versions :

```
public static boolean stop() {
    // Utilise ServerLifecycleHooks.getCurrentServer()
    // sans besoin de CommandContext
}
public static boolean stop(CommandContext ctx) {
    return stop(); // delegate
}
```

Ceci permet d'appeler `stop()` depuis `eliminateTeam()` sans contexte de commande.

4. Systeme de spectateur

Le systeme de spectateur gere les joueurs morts. Il a ete entierement reecrit avec 3 modes distincts.

4.1 Les trois modes

- TEAMMATE_WATCH : le joueur est en mode SPECTATOR, son point de vue est attache a un coequipier vivant. Utilise setCamera().
- BASE_SPECTATE : (NOUVEAU) le joueur est en mode ADVENTURE avec vol et noclip, confine a sa zone de base (FCZ). Utilise quand aucun coequipier n'est en vie.
- FREE_SPECTATE : le joueur est en mode SPECTATOR, peut voler partout sauf dans les bases ennemies.

4.2 Vol et noclip en mode ADVENTURE

Le mode ADVENTURE desactive normalement le vol par default. L'implementation ajoute :

```
// Dans enterBaseSpectate() :
spectator.noPhysics = true;
spectator.getAbilities().mayfly = true;
spectator.getAbilities().flying = true;
spectator.onUpdateAbilities();

// Dans tickBaseSpectate() (chaque seconde) :
if (!spectator.getAbilities().mayfly)
    spectator.getAbilities().mayfly = true; // reaffected
spectator.noPhysics = true;
```

Note : noPhysics est un champ protected de Entity. En 1.21.1, il n'y a pas de setter public. Le champ est directement affecte.

4.3 Phases de respawn

Phase 1 (jours 1-6) : les joueurs respawn automatiquement apres un timer = morts * 3 secondes (max 20s).

Phase 2 (jours 7-12) : les joueurs restent morts jusqu'a ce qu'un coequipier utilise un Totem de Revivality ou que le jour suivant commence.

handlePhase2DayStart() respawn les spectateurs dont le coeur d'equipe est encore intact. Ceux dont le coeur est detruit passent en FREE_SPECTATE.

4.4 Detection de destruction de coeur en cours de jour

Le tick handler (tick toutes les 20 ticks) detecte si le coeur de l'equipe a ete detruit en phase 2 et bascule automatiquement le joueur en FREE_SPECTATE avec un message.

4.5 FreeCamZone (FCZ)

Zone rectangulaire definie par deux coins (fcz_min, fcz_max) stockes dans TeamData. Le spectateur en BASE_SPECTATE est teleporte a l'interieur s'il sort de la zone. En l'absence de FCZ, un rayon de 30 blocs autour du spawn est utilise comme fallback.

4.6 ZoneVisualizer

Affichage de particules (DustParticleOptions) aux aretes de la zone toutes les 5 ticks. Utilise 8 couleurs d'equipe. Active par /xiidays zone show [equipe].

5. Popups de points

5.1 Principe

Quand un joueur gagne ou perd des points, au lieu d'un message dans le tchat, un popup apparait en bas a droite de l'ecran avec le montant et le type d'action (KILL, TOTEM, MINING, etc.).

5.2 Flux

```
1. PointsManager.addPoints() cree PointsPopupPayload
2. PacketDistributor.sendToPlayer(srvp, popup) envoie au joueur
3. PointsPopupPayload.handle() -> context.enqueueWork ->
   PointsPopupClientHandler.show(payload)
4. ClientModEvents a enregistre le calque GUI qui appelle
   PointsPopupClientHandler.render() a chaque frame
```

5.3 Animation

- Stocke les popups dans une liste, max 5 simultanées
- Duree de vie variable : 3s (1 popup) a ~10s (5 popups)
- Nouveau popup apparait 20px en dessous de sa position cible et glisse vers le haut (smooth)
- Quand un nouveau popup arrive, tous les popups existants glissent smooth vers le bas d'un cran (SMOOTHING = 0.12)
- Fondu dans les 600 dernieres ms
- Couleur : vert (#00FF00) si points >= 0, rouge (#FF0000) si negatif
- Panneau semi-transparent avec bordure coloree en haut

5.4 Chat conserve

Le message destine a l'equipe dans le tchat est conserve pour que les coequipiers voient les actions de chacun. Seul le message personnel (envoye au joueur uniquement) est remplace par le popup.

6. Totem de Revivality

6.1 Description

Item consommable qui permet de ressusciter un coéquipier mort en phase 2. Clique-droit sur le joueur spectateur pour le ramener à la vie.

6.2 Propriétés

- Item ID : xiidays:totem_revivalite
- Stack max : 16
- Rarity : RARE
- Model JSON et texture PNG fournis
- Traductions EN et FR
- Ajoute à l'onglet creative INGREDIENTS

6.3 Comportement

```
interactLivingEntity() :
1. Verifier cote serveur
2. Verifier que la cible est un ServerPlayer spectateur
3. Verifier que le joueur et la cible sont dans la meme equipe
4. Appeler SpectateManager.respawnPlayer(targetPlayer)
5. Si echec -> return FAIL
6. Ajouter 200 points à l'equipe :
   PointsManager.addPoints(teamId, PointType.TOTEM, serverPlayer)
7. Consommer l'item (sauf creative)
8. Jouer le son TOTEM_USE
9. Messages de confirmation
```

6.4 PointType.TOTEM

Le type TOTEM existe déjà dans l'enum PointType et attribue 200 points par défaut dans PointsManager.

7. Ecrans et interface

7.1 Design commun

Tous les ecrans (ScoreboardScreen, EndScoreboardScreen, TeamStatsScreen) suivent le meme design :

- Fond semi-transparent noir (#CC000000)
- Panneau central dark navy (#FF1A1A2E)
- Bordure haute or de 3px (#FFFFD700)
- Titres centres en or
- Texte en § (formatage Minecraft)
- Barre de scroll laterale or
- isPauseScreen = false (ne bloque pas le jeu)
- renderBlurredBackground vide (pas de flou)
- shouldCloseOnEsc = true

7.2 EndScoreboardScreen - Refonte

AVANT : lisait DataManager directement depuis le client (impossible -> plantage), textures externes (end_scoreboard_bg.png, mvp_frame.png, trophy.png) buggees ou absentes.

APRES : recoit les donnees pre-calculées du serveur via OpenEndScoreboardPacket, plus aucune texture ni reference a DataManager/TeamManager/RenderSystem.

Contenu :

- Banniere gagnant avec emoji -> et stats resume (pts/kills/morts)
- Section MVP avec etoile, nom, points contribues, equipe
- Classement complet avec medailles , , et details par equipe
- Scrollable si plus de place que le panneau

7.3 PointsPopupOverlay

Calque GUI enregistre via RegisterGuiLayersEvent, rendu au-dessus de tous les autres layers vanilla. Ne s'affiche pas si un ecran (Screen) est ouvert.

7.4 TeamStatsScreen - Correction

Ajout d'un override de renderBlurredBackground() vide pour eviter le double assombrissement du fond qui se produisait avec le comportement par default de Screen.

8. Commandes

8.1 Architecture

Toutes les commandes sont sous le root `/xiidays` avec un alias `/xd` via Brigadier `redirect()`. Un seul point d'entree dans `CommandRegistry` enregistre dans le `RegisterCommandsEvent`.

8.2 Sous-commandes

```
/xiidays day      start|stop|status|set <1-12>
/xiidays team     create|remove|add|spawn|core|eliminate
                  |revive|list          <- revive NOUVEAU
/xiidays spec     respawn|respawnall|list
/xiidays zone     set|remove|show|hide
/xiidays restrict status|items|blocks|bypass
/xiidays data     reload|save|domains
/xiidays score    (ouvre le scoreboard)
/xiidays (sans arg) affiche l'aide
```

8.3 Commande revive

NOUVELLE : `/xiidays team revive <equipe>`

- Permet de rehabiliter une equipe eliminee
- Suggestion automatique des noms d'equipes (`TEAM_MANAGER_SUGGESTIONS`)
- Permission : `op (4)`
- Message de confirmation : Equipe X rehabilitee !

8.4 Commande zone

Permet de definir, retirer et visualiser les `FreeCamZones (FCZ)` utilisees par le mode `BASE_SPECTATE`.

- `/xiidays zone set <equipe> <pos1> <pos2>` -> definit les deux coins
- `/xiidays zone remove <equipe>` -> supprime la zone
- `/xiidays zone show [equipe]` -> active le trace de particules
- `/xiidays zone hide` -> desactive le trace

9. Gestion des donnees

9.1 DataManager

Systeme de persistance via fichiers JSON. Chaque domaine est un fichier .json dans le repertoire xiidaysdata du monde.

- Domaine 'teams' -> teams.json : equipes, membres, config (eliminated, finalpos, spawn, core...)
- Domaine 'team_stats' -> team_stats.json : points, kills, killstreak, blocs mines, degats...
- Domaine 'player_stats' -> player_stats.json : team_points, deaths par joueur

9.2 Correction fuite entre mondes

PROBLEME : dataDir etait un champ static initialise une seule fois. En solo, changer de monde conservait le chemin de l'ancien monde.

SOLUTION :

- dataDir n'est plus mis en cache (calcule a chaque appel)
- Un champ cachedServer memorise la reference du serveur
- Si le serveur change, le cache JSON est vide (cache.clear())
- Ainsi, chaque monde a ses propres fichiers et son propre cache

9.3 API de compatibilite

Les methodes dataRead(), dataReadInt() et getAllDataNames() sont conservees dans DataManager pour la retrocompatibilite avec les ecrans qui les utilisent encore (bien que ces ecrans aient ete migres vers les paquets reseau).

9.4 TeamData - Ajouts

Les champs FCZ ont ete ajoutes a la configuration d'equipe :

- fcz_min (String) : coin min de la zone sous forme "x,y,z"
- fcz_max (String) : coin max de la zone

10. Corrections de bugs

1 EndScoreboardScreen non affiche

L'ecran lisait DataManager directement depuis le client (inexistant). Solution : pre-calcul des donnees sur le serveur, envoi via OpenEndScoreboardPacket.

2 Jour non arrete automatiquement

eliminateTeam() ne faisait que signaler le gagnant sans arreter le jour. Solution : appel a DaysManager.stop() avant envoi de l'ecran de fin.

3 Jour relancable apres fin de partie

Aucun verrou n'empechait DaysManager.start() apres la fin. Solution : ajout de isGameOver() check dans start(), bloque le demarrage si un gagnant existe.

4 Totem de Revivality sans points

L'item ressuscitait le joueur mais n'attribuait pas les 200 points TOTEM. Solution : appel a PointsManager.addPoints() apres respawnPlayer().

5 Fuites de donnees entre mondes solo

DataManager statique conservait le chemin du monde precedent. Solution : detection du changement de serveur + recalcul du chemin + vidage du cache.

6 Textures bugges EndScoreboardScreen

L'ecran utilisait des textures externes non fonctionnelles. Solution : redesign complet sans textures, style panneau dark navy unifie.

7 Spectateur sans vol en base

Le mode ADVENTURE desactive le vol par default. Solution : reaffectation de mayfly et noPhysics a l'entree et maintien tick.

8 RestrictionsManager plantage client

onRightClick/onAttackEntity appelaient level() sans verifier isClientSide. Solution : ajout de guards isClientSide().

9 TeamStatsScreen double assombrissement

Le fond etait assombri deux fois (par Screen + par le rendu custom). Solution : override renderBlurredBackground vide.

10 TeamData.setFreeCamZoneMin/Max plantait sur null

addProperty(null, ...) leve une exception. Solution : utilisation de remove() quand la valeur est null.

11. Decisions d'architecture

Paquets réseau pour les données d'écran

Toutes les données d'écran (scoreboard, end-scoreboard, team stats) transitent par des paquets S2C pré-calculés sur le serveur. Le client ne lit jamais les fichiers JSON directement. Ceci évite les problèmes de synchronisation et les crashes côté client.

StreamCodec manuel plutôt que NetworkCodec

Les StreamCodec sont écrits manuellement pour chaque paquet plutôt que générés automatiquement. Cela donne un contrôle total sur la sérialisation et évite les surprises avec les nouvelles versions.

3 modes spectateur distincts

TEAMMATE_WATCH (attache à un joueur), BASE_SPECTATE (confinement zone + survol), FREE_SPECTATE (vol libre sans accès aux bases). Cette séparation permet des comportements différents et des contraintes adaptées à chaque situation.

Maintien tick des capacités du spectateur

Le mode ADVENTURE réinitialise les capacités (mayfly, noPhysics) à chaque tick. On les réaffecte donc dans le tick handler pour garantir leur persistance.

ADVENTURE plutôt que SPECTATOR pour BASE_SPECTATE

Le mode SPECTATOR permettrait de traverser le sol et de voir les autres joueurs de manière non prévue. ADVENTURE + invulnerable + noPhysics empêche le clipping tout en autorisant le vol.

Animation smooth par interpolation exponentielle

Les popups utilisent un lissage exponentiel ($\text{visualY} += (\text{target} - \text{visualY}) * 0.12$) pour des transitions fluides sans gestion d'état complexe.

Durée de vie des popups variable

La durée d'affichage augmente avec le nombre de popups actifs (base 3s + 1.5s par popup supplémentaire, max 10s) pour laisser le temps de tout lire.

Reset partiel du gagnant lors de la réhabilitation

Quand une équipe est réhabilitée, le statut de vainqueur ($\text{finalPosition} == 1$) est effacé pour toutes les équipes. Sinon `isGameOver()` resterait vrai et empêcherait la reprise.

Calcul des morts d'équipe à la volée

`TeamStatsData` n'a pas de champ `deaths`. Les morts d'équipe sont calculés au moment de l'envoi du paquet de fin en sommant `PlayerStatsData.getDeaths()` de chaque membre. Évite la duplication de données.

DataManager sans cache de chemin

Le chemin du répertoire de données est recalculé à chaque accès (avec vérification du changement de serveur). Impact négligeable en performance (simple résolution de chemin) et supprime un bug source de bugs.

12. Liste exhaustive des fichiers

12.1 Fichiers crees

- network/OpenEndScoreboardPacket.java
- network/ClientEndScoreboardHandler.java
- network/PointsPopupPayload.java
- network/PointsPopupClientHandler.java
- client/ClientModEvents.java
- item/TotemRevivalityItem.java

12.2 Fichiers modifies

- network/PacketHandler.java
- screen/EndScoreboardScreen.java
- screen/ScoreboardScreen.java
- screen/TeamStatsScreen.java
- game/TeamManager.java
- game/DaysManager.java
- game/PointsManager.java
- spectator/SpectateManager.java
- commands/XiidsaysCommand.java
- commands/CommandRegistry.java
- data/DataManager.java
- data/TeamData.java

12.3 Fichiers renommes/deplaces

- blocks/teamCore -> world/CoreBlockEntity
- blocks/teamSpawner -> world/SpawnerBlockEntity
- blocks/TeamCoreBlock -> world/CoreBlock
- blocks/TeamSpawnerBlock -> world/SpawnerBlock
- blocks/BlockRegistry -> world/BlockRegistry
- blocks/BlockEntityRegistry -> world/BlockEntityRegistry
- utils/DataManager -> data/DataManager
- utils/TeamData -> data/TeamData
- utils/TeamStatsData -> data/TeamStatsData
- utils/PlayerStatsData -> data/PlayerStatsData
- utils/DayCycleData -> data/DayCycleData
- menus/ScoreboardScreen -> screen/ScoreboardScreen
- menus/EndScoreboardScreen -> screen/EndScoreboardScreen
- menus/TeamStatsScreen -> screen/TeamStatsScreen
- menus/MenuRegistry -> screen/MenuRegistry
- items/ItemRegistry -> item/ItemRegistry
- items/CoreDestroyerItem -> item/CoreDestroyerItem
- blocks/ZoneVisualizer -> visual/ZoneVisualizer
- utils/Config -> config/Config
- RestrictionsManager -> restriction/RestrictionsManager

- SpectateManager -> spectator/SpectateManager
- PlayersHandler -> player/PlayersHandler

12.4 Fichiers supprimes

- utils/SpectatorZoneCommand.java (remplace par /xiidays zone)
- utils/DataManager.java (duplicata, un seul garde dans data/)

Fin du document. Toutes les modifications sont compilees dans le repertoire source src/main/java/com/mceteams/xiidays/ du projet Minecraft XII Days.